SC-RAG

面向实习求职的随机计算文献研究 Agent

项目实现、评测体系、系统原理与面试准备指南

> 这份文件是个人学习与面试准备材料，不属于公开 GitHub 仓库。

一页讲清项目

SC-RAG 是一个面向 stochastic computing 研究的 evidence-grounded literature agent。它把 50 篇论文转换为带页码和稳定 chunk ID 的证据，通过中英文混合检索和 alias-aware evidence planner 找到论文级证据，再让模型生成带 Claim-level citation 的回答。应用层会验证 span、chunk、论文别名和原文摘录；证据不足、比较不完整或模型输出不符合 schema 时，系统拒答。

项目的差异点不在“接入了一个大模型”，而在于把研究问答拆成可测量、可审计、可失败归因的链路：检索质量、证据覆盖、生成合同、引用正确性、拒答行为和延迟都能单独评估。

面试时可以用下面这段 60 秒介绍：

我做了一个面向随机计算论文研究的 evidence-grounded RAG agent。系统从 50 篇 PDF 建立页级索引，使用 BM25 与 multilingual dense retrieval 做 RRF 融合；对于用户明确指定的论文，系统会按 alias 单独补召回并优先规划证据。LLM 只能引用应用生成的 span ID，最终引用由服务端根据可信检索元数据解析，因此能防止模型伪造论文、页码和摘录。除了单轮问答，我还做了两遍式文献综述：先逐篇抽取方法、位流、硬件、实验和局限，再基于已验证事实做跨论文比较和可证伪研究假设。最后用 answerable/unanswerable benchmark、引用合同、拒答率、延迟和失败归因做回归。

一、系统架构与请求工作流

系统分成离线数据管线、在线检索与回答、文献综述和评测可观测性四个层次。

层次	职责	关键代码
数据	PDF → manifest → page-aware chunks；References 截断。	src/sc_rag/ingest.py
检索	BM25、中文/英文术语扩展、dense、RRF、论文别名定向搜索。	search.py / dense.py / hybrid.py
生成	结构化 JSON、Claim、span ID、repair、拒答。	answering.py
研究工作流	逐篇事实抽取 → 事实 ID 绑定 → 跨论文综合。	literature_review.py
产品与评测	FastAPI、Streamlit、回归报告、延迟与失败归因。	api.py / ui/app.py / scripts

在线问答的 10 个步骤

1. ■■ question ■ AnswerMode
2. HybridRetriever.search(question, k=30)
3. ■■■■■■■■■■ alias
4. ■■■ alias ■■ search_alias ■■■■■
5. plan_evidence■■■■■■■■■■■■■■■■■■
6. split_evidence_spans■■■ E1.S1 ■■■ span ID
7. LLM ■■■ untrusted source text ■■■ metadata
8. RawAnswer Pydantic ■■■■■■■ repair ■■
9. ■■■■ span ID ■■■■■ chunk■■■■■■
10. ■■■■■■■■■■■■■■■■ fail closed

为什么“ 模型生成引用 ”是不可信的？

模型可以生成看起来合理的论文名、页码甚至 quote。SC-RAG 不让模型决定引用的 paper_id、page 或 excerpt，而只允许模型返回 E1.S1 这样的内部标识。应用从本次请求的证据计划中解析该标识，使用检索元数据生成最终引用。这是一个典型的“ 模型提议、应用裁决 ”边界。

二、离线数据管线与检索原理

PDF 处理

manifest 保存 paper_id、alias、title、relative_path 和 hash；ingest 使用 PyMuPDF 逐页提取文本，保留 PDF page。chunk ID 由 paper、page、local index 和稳定 hash 组成，因此评测和引用可以复现。References/Bibliography /参考文献标题之后的内容会被截断，避免文献列表成为虚假正文证据。

```
make manifest
make ingest
make dense-index EMBEDDING_MODEL=intfloat/multilingual-e5-small
```

BM25 与 dense 的互补性

BM25 对术语、缩写、型号和论文别名很敏感，延迟低；dense retrieval
对语义改写和中英文表达更有优势，但可能把“看起来相似”的参考文献或背景段落排得很前。RRF
不直接比较分数，而是按各自 rank 融合：

RRF(d) = Σ 1 / (k + rank_i(d))。在项目中，hybrid 的目标不是平均两个检索器，而是提升 evidence coverage，同时使用 max_per_paper 控制单篇论文占满上下文。

为什么要有 alias-aware planner？

标准 top-k 可能全部来自一篇高频论文。对于“比较 [A] 和 [B]”这类问题，系统先解析显式 alias，再对每个 alias 做定向搜索并优先放入证据。这样把“目标论文是否进入证据”和“模型是否引用目标论文”拆成两个可诊断指标。

三、Grounded Generation 与防幻觉机制

结构化输出合同

```
{"evidence_sufficiency":"sufficient|partial|insufficient|conflicting",
 "confidence":0.0,
 "claims":[{"text":"...","citations":[{"span_id":"E1.S1"}]}],
 "missing_information":[], "reason":"..."}
```

RawAnswer、RawClaim 和 RawCitation 使用 extra=forbid。模型多返回字段会进入 repair；repair 后仍非法则拒答。citation 的最终 metadata 不来自模型，而来自 by_span_id。

三类重要门控

- 证据不足：evidence_sufficiency=insufficient 且没有可支持 Claim 时 abstained。
- 比较不完整：comparison 模式的 partial 至少要引用两个不同来源；只有一方数据时拒答。
- 目标论文漏引：目标 alias 已在证据中但没有出现在 citation 时触发一次 repair；修复后仍不满足则不接受。

为什么不能简单让 LLM-as-a-judge 判分？

LLM judge 自己也可能受答案措辞、引用幻觉和评测泄漏影响。SC-RAG 先用确定性合同检查 span、chunk、alias、quote 和状态，再把人工审核用于语义支持性和可替代来源。自动指标保证一致性，人工审核补足语义判断。

四、两遍式文献综述 Agent

Pass 1：逐篇事实抽取

每篇论文独立输入有限证据，抽取 method、bitstream、hardware、evaluation、limitations 五类事实。服务端为每个事实绑定 F1、F2 等 fact_id 和真实 Citation，禁止后续综合阶段引用未验证的事实。

Pass 2：跨论文综合

综合模型只接收已经绑定 fact_id 的事实表，而不是重新接收全部 PDF。ComparisonCell 必须引用 fact_ids；ResearchIdea 必须包含 observed_gap、hypothesis、proposed_experiment、falsifier 和 uncertainty。这样“研究想法”与论文事实之间保留了可审计边界。

```
POST /literature-review
{"topic":"[redacted]",
 "aliases":["Sobol","PSobol","P2LSG","FSM-LDSG"],
 "chunks_per_paper":5}
```

为什么这比一次性长 Prompt 更好？

一次性把 4 篇或 50 篇论文塞进上下文，会导致 token 成本、注意力稀释、来源混淆和不可定位错误。两遍式流程把“抽取”和“综合”分开，事实 ID 作为中间表示，既节省上下文，也让失败可以定位到具体论文或具体 synthesis cell。

五、评测体系与真实结果解读

评测分层

层级	指标	回答的问题
Retrieval	hit_any、hit_all、MRR、p95 latency	目标论文和相关证据是否被找全？
Evidence planning	explicit_target_evidence_hit_all	目标论文是否进入实际上下文？
Generation	answered/partial/abstained、repair rate	模型是否覆盖、是否过度拒答？
Citation	citation contract、gold alias recall	引用是否来自真实证据？
Safety/Gating	unanswerable abstention rate	没有证据时是否拒答？

项目中一次重要的归因

Phase 4 人工审核发现：E1 把 References 当正文，E5 在比较任务只找到一方仍返回 partial，D4 的正文实验环境被 References 切片挤出。修复分别落在三个层级：ingest 截断参考文献、comparison partial 双来源门控、索引重建。这是面试中很好的“从指标到根因再到代码修改”的闭环。

如何运行完整回归

```
make evaluate-phase4
# artifacts/phase4_eval.json
python scripts/generate_phase4_human_review.py
# artifacts/phase4_human_review.md
```

Phase 4 报告会保存候选数、证据别名、目标 alias、模型状态、repair、延迟和失败分类，避免只报一个总分。

六、面试问题与专业回答

Q1：为什么不用纯向量检索？

随机计算论文包含大量缩写、硬件型号、算法名和公式符号。BM25 对精确词和实体召回更稳定，dense 对语义改写更好。我们先分别评估，再用 RRF 做融合，并通过 max_per_paper 和 alias planner 保证跨论文覆盖。

Q2：你的 RAG 如何防止幻觉引用？

模型只能返回 span_id；应用根据本次 evidence plan 解析 chunk、paper、page 和 excerpt。未知 span、重复引用、quote 不在证据或 schema 错误都会 fail closed。最终引用字段不是模型自由生成的。

Q3：为什么 partial answer 有时也要拒答？

对开放式基础问题，partial 可以保留已支持事实并列出缺失信息；但 comparison 问题的核心任务是对照，只有一方证据时 partial 容易被误读为完成比较。因此 comparison 模式要求至少两个来源的 citation，否则拒答。门控规则应该由任务语义决定。

Q4：如何证明 RAG 真的有效？

不能只展示一条漂亮回答。我们固定 50 answerable + 8 unanswerable benchmark，分别报告检索命中、显式目标证据覆盖、引用合同、可用响应率、拒答率、repair rate 和延迟；再对失败样本做 retrieval/generation/gating 归因。

Q5：LLM-as-judge 的局限是什么？

它会受到答案风格、模型偏好和引用幻觉影响，且难以稳定判断 quote 是否真的来自证据。因此确定性检查负责边界合同，人工审核负责语义支持性；judge 只能作为辅助指标。

Q6：如果扩展到百万篇论文，哪里会先成为瓶颈？

离线 embedding 和索引构建、在线 candidate fan-out、reranker 延迟和上下文成本会先出现问题。需要分层索引、租户/领域过滤、缓存 query embedding、限制每篇论文贡献、异步综述任务以及 trace-based cost budget。

Q7：你如何定位一次失败？

先看 trace：candidate_count 是否足够、evidence_aliases 是否覆盖目标、missing_requested_aliases 是否为空；再看 generation status、repair_reason 和 citation contract。如果目标未进证据，是 retrieval/planning；进入证据但未引用，是 generation；引用字段非法，是 validation；不可回答题被回答，则是 gating。

七、Codespaces 演示操作手册

启动前检查

```
cd /workspaces/SC-RAG
git pull origin main
make test
curl http://127.0.0.1:8000/health
```

8501 是 Streamlit 页面端口，8000 是 FastAPI
服务端口。两个服务必须分别在两个终端以前台方式启动。不要只打开 8501；如果 8000
没有服务，页面会显示 Connection refused。

标准启动方式

```
■■ 1■
cd /workspaces/SC-RAG
make api PORT=8000

■■ 2■
cd /workspaces/SC-RAG
make setup-ui
export SC_RAG_API_URL=http://127.0.0.1:8000
make ui
```

在 Codespaces 的 Ports 面板中打开 8501。先在终端 1 用 curl 验证 /health，再打开 UI。终端 1 不能关闭；关闭后 8501 仍可能显示页面，但所有请求都会失败。

端口被占用时

```
curl http://127.0.0.1:8000/health

# ■■ 8000 ■■■■■■■■
make api PORT=8001

# ■■■■■■■■■■■■■■■■
export SC_RAG_API_URL=http://127.0.0.1:8001
make ui
```

面试演示顺序

- 打开页面后先做一个基础知识问题，展开 Claim 查看 alias、PDF 页码、span ID 和 excerpt。
- 输入“比较 [Sobol] 和 [P2LSG] 的并行化方法”，展示两篇论文都进入证据。
- 输入语料不支持的问题，展示 abstained、missing information 和拒答原因。
- 切换 Literature Review，输入主题和至少两个 alias，展示 comparison 与 research idea。
- 最后打开 artifacts/phase4_eval.json，解释一次失败如何归因到 retrieval、generation 或 gating。

七、简历项目描述与演示脚本

简历中文版本

设计并实现面向随机计算文献研究的 evidence-grounded RAG Agent：基于 50 篇论文构建页级可引用语料，融合 BM25 与 multilingual dense retrieval，并通过 alias-aware evidence planner
保证跨论文任务的目标来源覆盖；实现结构化 Claim-level citation、span 级引用校验、模型输出 repair 和 fail-closed abstention。进一步实现两遍式文献综述
Agent，支持逐篇事实抽取、跨论文比较和带证伪条件的研究假设生成；构建 answerable/unanswerable 回归集、引用合同、延迟和失败归因评测链路，并提供 FastAPI + Streamlit 演示工作台。

两分钟演示顺序

- 先问一个基础知识问题，展开一条 Claim 的 PDF 页码和摘录。
- 再输入“ 比较 [Sobol] 和 [P2LSG] ”，展示两篇论文都进入证据。
- 输入一个语料没有答案的问题，展示 abstained 和缺失原因。
- 打开 Literature Review，展示逐篇事实、comparison cell 和 research idea。
- 最后打开 phase4_eval.json，说明你如何把失败归因到检索、生成或门控。

最后检查清单

- README 能在新 Codespace 按步骤运行，不依赖私人教程。
- 公开仓库不含 raw PDFs、API key、生成报告和私有学习笔记。
- 测试、评测、UI 和 API 命令都能复现。
- 至少准备 E1、E5、D4 三个真实失败案例的根因和修复。
- 能解释每个指标为什么存在，而不是只背结果。